Introdução à Visão Computacional e Redes Convolucionais

Aula 06 — Da Ruptura do Flattening à Geometria dos Filtros 2D no PyTorch

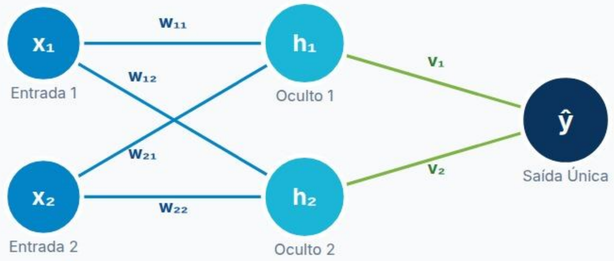
Instrutor: **Prof. Allan Spadini**

Redes Neurais Profundas | Graduação Faculdade Infnet

TOPOLOGIA CLÁSSICA (2 → 2 → 1)

Perceptron Multicamadas (MLP) Simples

1. Entradas | 2. Oculta | 3. Saída | ⚡ Simular Forward



Entradas (2) | Camada Oculta (2) | Saída Estimada (1)

CAMADA DE SAÍDA (1 NEURÔNIO)

Neurônio de Saída ŷ

Pega as respostas de h_1 e h_2, multiplica pelos pesos v_1 e v_2, soma o bias b_o e aplica f para dar a resposta final estimada ŷ.

EXPRESSÃO MATEMÁTICA LOCAL:

$$\hat{y} = f(v_1 \cdot h_1 + v_2 \cdot h_2 + b_o)$$

💡 **Conceito Central:** Em uma rede densa com 2 entradas e 1 saída, todas as conexões entre camadas vizinhas possuem pesos livres ajustados durante o treinamento.

Como as Contas se Conectam: O Forward Pass

Apenas três passos: multiplicar pelo peso, somar o bias e passar pela função f

$$h_1 = f(w_{11} \cdot x_1 + w_{21} \cdot x_2 + b_1)$$

$$h_2 = f(w_{12} \cdot x_1 + w_{22} \cdot x_2 + b_2)$$

$$\hat{y} = f(v_1 \cdot h_1 + v_2 \cdot h_2 + b_o)$$

x_1, x_2

As 2 entradas que chegam na rede (ex: duas medições ou dois pixels).

`w, v`

Pesos multiplicadores: a importância que a rede dá a cada conexão.

`b`

Bias (viés): número somado para deslocar a reta de decisão.

$f(\cdot)$

Função de ativação: transforma o valor obtido (ex: zera valores negativos).

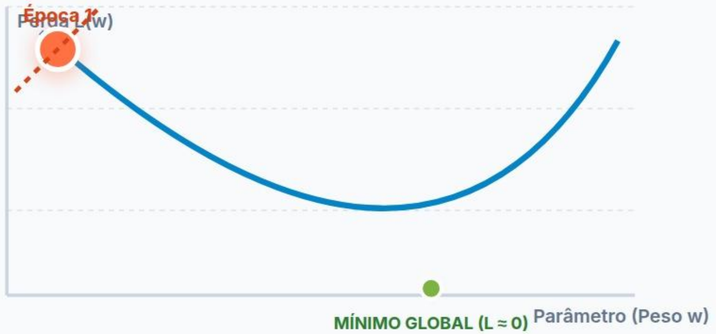
$\hat{y}$

O resultado final estimado pela rede para aquela amostra.

SUPERFÍCIE DE PERDA E PONTO DE MÍNIMO

Minimização da Função Objetivo por Época

Reiniciar | Próxima Época →



Época 1 | Época 2 | Época 5 | Época 10 | Época 25

A PERDA É A DIFERENÇA (RESÍDUO)

FUNÇÃO DE PERDA (DIFERENÇA AO QUADRADO):

$$Perda = (y - \hat{y})^2$$

(Valor Real no Treino – Valor Estimado pela Rede)²

🟢 **Conhecido no Treino (Gabarito y):** 1.00

🔵 **Estimado pelo Modelo (Predição ŷ):** 0.15

Diferença (y - ŷ): 0.85 → Loss: 0.3610

Início do treino: predição distante do gabarito real. Erro e perda elevados.

🎯 **Regra de Ouro:** Treinar a rede é ajustar os pesos de modo que a diferença entre o que se conhece (treino) e o que se estima (predição) convirja para zero.

Como os Pesos Aprendem: O Backpropagation

Ajuste do peso pelo erro com cálculo formal da derivada no canto separado

INTUIÇÃO CENTRAL DO TREINAMENTO

Como os Pesos Aprendem a Cada Época

1. DIFERENÇA DIRETA (ERRO DA REDE)

$$e = y - \hat{y}$$

Erro = (Valor Conhecido no Treino) − (Valor Estimado pela Rede)

2. ATUALIZAÇÃO DO PESO

$$w_{\text{novo}} = w_{\text{antigo}} + \Delta w$$

O peso novo é o peso anterior corrigido pelo ajuste Δw

$y - \hat{y}$	$\eta\ (\text{LR})$	w_{novo}
Se o erro for zero, o peso não muda.	Taxa de aprendizado: tamanho do passo.	Reduz o erro na tentativa seguinte.

📐 CANTO DE CÁLCULO FORMAL

Como o Ajuste (Δw) é Calculado?

O ajuste não é um chute arbitrário: ele é derivado formalmente pela **Regra da Cadeia** a partir da perda:

FÓRMULA EXATA DO AJUSTE:

$$\Delta w = -\eta \cdot \frac{\partial L}{\partial w}$$

$$\Delta w = \eta \cdot (y - \hat{y}) \cdot f'(z) \cdot x$$

$(y - \hat{y})$	Tamanho e sinal do erro cometido na saída.
$f'(z)$	Derivada da ativação: modula a sensibilidade do neurônio.
x	Intensidade da entrada que participou da conexão.

⚙️ **No PyTorch: O mecanismo do `loss.backward()` calcula essa derivada para milhões de conexões automaticamente!**

O Que é Uma Imagem Digital?

Uma matriz bidimensional de intensidades luminosas

VISÃO HUMANA

Percepção Visual Contínua

- Enxergamos formas, contornos, objetos e rostos.
- Percebemos imediatamente a relação entre pontos vizinhos.
- Reconhecemos um padrão mesmo que ele mude de posição.

VISÃO DA MÁQUINA

Matriz Numérica Discreta

- Uma grade 2D de números inteiros entre 0 (preto) e 255 (branco).
- Em PyTorch, convertemos para tensores float no intervalo [0.0, 1.0].
- Formato em escala de cinza: (1, Altura, Largura).

Explorador Interativo de Matrizes de Pixels

Passe o mouse sobre os pixels para ver seus valores numéricos e o efeito do Flattening

VISUALIZADOR INTERATIVO

Matriz de Pixels 0-255 & O Efeito do Flattening (Estiramento)

🖼️ Visão Matricial 2D (6x6) | 📏 Vetor Estirado 1D (36)

Matriz de Imagem 2D (6×6 pixels)

20	20	255	255	20	20
20	200	255	255	200	20
20	200	20	20	200	20
20	200	20	20	200	20
20	200	255	255	200	20
20	20	255	255	20	20

🔍 Detalhes do Pixel

Passe o cursor sobre qualquer célula da matriz para inspecionar seu valor de brilho e coordenadas.

Intuição Unidimensional da Convolução

Como um filtro deslizante detecta padrões locais através de pesos compartilhados

SIMULADOR INTERATIVO 1D

Convolução 1D, Filtros & Redução de Amostras

▶️ Animar Convolução | 🔄 Reiniciar

Selecione o Filtro: **Detecção de Borda [-1, 0, 1]** | Suavização [0.33, 0.33, 0.33] | Realce / Aguçamento [-1, 2, -1]

Passo Atual: **1 / 12**

Sinal de Entrada ($N = 14$ amostras) — **Janela Kernel $K = 3$ em vermelho**

10 [0]	10 [1]	10 [2]	10 [3]	80 [4]	80 [5]	80 [6]	20 [7]	20 [8]	20 [9]	90 [10]	90 [11]	10 [12]	10 [13]

Sinal Convoluído / Feature Map ($N_{out} = 12$ amostras) — **Redução de Amostras: 14 → 12**

0 [0]	0 [1]	70 [2]	70 [3]	0 [4]	-60 [5]	-60 [6]	0 [7]	70 [8]	70 [9]	-80 [10]	-80 [11]

🧮 Cálculo do Produto Escalar

Detecta variações abruptas de intensidade.

```
(10 × -1) = -10.0
(10 × 0) = 0.0
(10 × 1) = 10.0

Resultado = 0
```

Fórmula do Tamanho da Saída:

```
N_out = N_in - K + 1
12 = 14 - 3 + 1
```

A Matemática da Convolução 2D

O produto interno deslizante entre o kernel e o campo receptivo local

$$(I * K)(i,j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I(i+m,\ j+n) \cdot K(m,n)$$

I

Matriz da imagem de entrada (ou mapa de ativações da camada anterior).

K

Filtro (Kernel) convolucional com pesos aprendidos pelo backpropagation.

(i,j)

Coordenada espacial no mapa de características de saída.

Inductive Bias

Suposição arquitetural de que pixels próximos contêm informações correlacionadas.

Simulador Passo a Passo da Convolução 2D

Acompanhe o kernel deslizando sobre a matriz e gerando o feature map

SIMULADOR MATRICIAL 2D

Operação de Convolução 2D Passo a Passo

▶️ Animar Convolução 2D | 🔄 Resetar

Filtro (Kernel 3×3): Sobel Vertical (Detecção de Linha Vertical) | Sobel Horizontal (Detecção de Linha Horizontal) | Detecção de Bordas Gerais (Laplaciano)

Posição: Linha **0**, Coluna **0** (Passo 1/9) ◀ ▶

Entrada Imagem (5×5)

10	10	10	100	100
10	10	10	100	100
10	10	10	100	100
10	10	10	100	100
10	10	10	100	100

*

Kernel / Filtro (3×3)

-1	0	1
-2	0	2
-1	0	1

➔

Feature Map Resultante (3×3)

0	?	?
?	?	?
?	?	?

Soma dos Produtos na Janela Atual: (10×-1) + (10×0) + (10×1) + (10×-2) + (10×0) + (10×2) + (10×-1) + (10×0) + (10×1) = 0

Kernels Clássicos de Processamento de Imagens

Filtros fixos que a rede convolucional aprende a construir automaticamente

DETECÇÃO DE BORDAS

Filtros de Sobel

- Sobel Horizontal: detecta variações de luz de cima para baixo.
- Sobel Vertical: detecta transições de luz da esquerda para a direita.
- Realça contornos e formas geométricas essenciais dos objetos.

TRANSFORMAÇÃO ESPACIAL

Nitidez e Relevo

- Filtro de Nitidez (Sharpen): amplifica diferenças locais de contraste.
- Filtro Gaussiano (Blur): suaviza ruídos e reduz detalhes em alta frequência.
- Em PyTorch, a rede aprende os números do kernel automaticamente via SGD!

Cálculo da Dimensão de Saída Convolucional

A fórmula geométrica para determinar o tamanho dos mapas de características

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

W

Largura ou altura da entrada espacial (ex: 28 no MNIST).

K

Tamanho do filtro / kernel (ex: 3 para um kernel 3×3).

P

Preenchimento com zeros nas bordas (Padding).

S

Passo do deslizamento do kernel (Stride).

Padding (Preenchimento) & Stride (Passo)

Controlando as fronteiras e a velocidade de redução espacial

PADDING (PREENCHIMENTO COM ZEROS)

Preservando Bordas (Same Padding)

- Padding = 0 (Válido): a imagem encolhe a cada camada convolucional.
- Padding = (K-1)/2 (Same): a saída tem EXATAMENTE o mesmo tamanho da entrada.
- Garante que pixels nas bordas e cantos participem do mesmo número de convoluções.

STRIDE (PASSO DE DESLIZAMENTO)

Subamostragem Acelerada

- Stride = 1: o kernel anda de 1 em 1 pixel (resolução máxima).
- Stride = 2: o kernel pula de 2 em 2 pixels, cortando a resolução pela metade.
- Substitui camadas de pooling em arquiteturas mais modernas (ex: ResNets).

Simulador Interativo: Max Pooling vs. Average Pooling

Compare a extração da ativação mais forte contra a suavização pela média

SUBAMOSTRAGEM & INVARIÂNCIA

Simulador de Pooling (Max Pooling vs Average Pooling 2×2, Stride 2)

🔥 Max Pooling (Máximo) | ⚖️ Average Pooling (Média)

Selecione a Região de Pooling 2×2:

Quadrante 1 (Sup. Esquerdo) | Quadrante 2 (Sup. Direito) | Quadrante 3 (Inf. Esquerdo) | Quadrante 4 (Inf. Direito)

Feature Map de Entrada (4×4)

12	20	30	0
8	85	4	16
50	2	90	75
14	44	28	60

➔

Saída Subamostrada (2×2)

85	30
50	90

📊 Operação na Região Atual

Valores na janela 2×2: **[12, 20, 8, 85]**

`Max([12, 20, 8, 85]) = 85`

Retém a característica mais forte (maior ativação) e descarta o restante.

Por Que o Max Pooling Domina?

Compressão dimensional e conquista da invariância translacional

INVARIÂNCIA ESPACIAL

Tolerância a Pequenos Deslocamentos

- Se o objeto na imagem se deslocar 1 ou 2 pixels, o valor máximo ainda é capturado.
- Permite reconhecer dígitos ou objetos com leves variações de escrita e ângulo.
- Cria uma hierarquia: camadas profundas enxergam regiões cada vez maiores.

COMPRESSÃO EFICIENTE

Redução de Carga Computacional

- Um pooling 2×2 reduz a área da imagem em 75% (corta altura e largura pela metade).
- Diminui drasticamente o número de conexões para as camadas seguintes.
- Não possui nenhum parâmetro treinável (custo zero de pesos aprendidos).

A Arquitetura da Primeira CNN em PyTorch

Composição de blocos Conv2D, Ativações e Pooling para o MNIST

EXTRATOR DE CARACTERÍSTICAS

Bloco Convolucional

- nn.Conv2d(1, 16, kernel_size=3, padding=1): de 1 para 16 feature maps.
- nn.ReLU(): ativação não-linear para cada pixel do feature map.
- nn.MaxPool2d(2, 2): reduz resolução de 28×28 para 14×14.
- nn.Conv2d(16, 32, kernel_size=3, padding=1) + MaxPool2d: reduz para 7×7.

CLASSIFICADOR DENSO

Transição para nn.Linear

- Dimensão resultante: 32 canais de tamanho 7×7 = 1.568 valores.
- nn.Flatten(): lineariza apenas as representações abstratas profundas.
- nn.Linear(1568, 64) → nn.ReLU() → nn.Linear(64, 10).
- Atinge > 98.5% de acurácia no MNIST com uma fração dos pesos da MLP!

CONSOLIDAÇÃO DE CONHECIMENTO

Quiz Interativo: Visão Computacional & CNNs

✅ Enviar Respostas

C) O tamanho da imagem é duplicado.

D) A imagem é convertida em um vetor com 3 canais de cor.

2. Dada uma entrada 1D de tamanho N = 10 e um Kernel de tamanho K = 3 (com stride 1 e sem padding), quantas amostras terá a saída?

A) 10 amostras

B) 8 amostras (N - K + 1 = 10 - 3 + 1)

C) 7 amostras

D) 30 amostras

3. No PyTorch, qual é o formato correto do Tensor de uma imagem colorida RGB?

A) (Height, Width, Channels)

B) (Channels, Height, Width)

C) (Height * Width, Channels)

D) (Channels, Height * Width)

4. Qual é a principal função de uma camada de Max Pooling 2×2 com stride 2?

A) Aumentar a resolução da imagem para HD.

B) Reduzir o número de parâmetros ajustáveis na camada convolucional.

Síntese da Aula & Conexão com o Projeto 2

Base para a Análise de Alternativas Arquiteturais

PASSO 01

Localidade Espacial

Convoluções exploram a correlação 2D entre pixels vizinhos via receptive field.

PASSO 02

Parameter Sharing

O mesmo filtro desliza sobre toda a imagem, economizando milhões de pesos.

PASSO 03

Invariância Translacional

Max Pooling garante reconhecimento mesmo com pequenos deslocamentos espaciais.

PASSO 04

Trade-off MLP vs CNN

Fundamentação completa para o ensaio comparativo exigido no Projeto 2.

Próximos Passos: Aula 07

Otimização Avançada, Diagnóstico de Gradientes e TensorBoard

PASSO 01

Família de Otimizadores

Do SGD com Momentum ao AdamW com Weight Decay desacoplado.

PASSO 02

LR Schedulers

Agendamento dinâmico da taxa de aprendizado em platôs.

PASSO 03

Gradient Norms & Clipping

Diagnóstico numérico de gradientes para treinar MLPs e CNNs profundas.

PASSO 04

TensorBoard & Checkpoints

Monitoramento em tempo real com SummaryWriter e salvamento via state_dict.